



Wallet Control

Auditoría técnica, hallazgos de seguridad y plan a versión 1.0

— Actualización de avance: Fase 1 cerrada + Fase 2 cerrada + nuevas funcionalidades —

Preparado con Claude Code · claude-workspace
2 de julio de 2026 · actualizado 6 de julio de 2026

Contexto

Wallet Control es una app de finanzas personales con asesor de IA integrado ("Finando"), construida en Expo/React Native. Este documento resume la auditoría técnica original del 2 de julio de 2026 y su evolución hasta el 6 de julio, con la Fase 1 (bloqueantes) y la Fase 2 (pulido) ya cerradas, más varias funcionalidades nuevas construidas en el camino que no estaban en el alcance original.

Estado al 6 de julio de 2026: Fase 1 completa (7/7) · Fase 2 completa (todos los puntos accionables resueltos)

Ya se puede intentar un primer build de prueba. Solo falta un paso manual del usuario: correr `eas login + eas init` para vincular el proyecto a una cuenta EAS antes de `eas build --profile preview` (ver NOTAS/ajustes `wallet-control.txt`).

Hallazgos de la auditoría original — Fase 1 (todos resueltos)

Seguridad — antes de compartir un build con IA real

- RESUELTO 1. API key de Anthropic expuesta en el bundle**
Se implementó un backend real: la llamada a Claude ahora pasa por una Supabase Edge Function (`supabase/functions/claude-proxy` , `verify_jwt:true`) con la API key como secreto de servidor. `lib/claude.ts` ya no lee `EXPO_PUBLIC_ANTHROPIC_API_KEY` ni llama directo a `api.anthropic.com` .
- RESUELTO 2. Contraseñas en texto plano**
El campo `password` fue eliminado por completo de `lib/storage.ts` ; el login real ahora usa `supabase.auth` .
- RESUELTO 3. Modelo desactualizado**
`lib/claude.ts` usa `claude-sonnet-5` .

Build y publicación

- RESUELTO 4. ios.bundleIdentifier y android.package**
Presentes en `app.json` (`com.walletcontrol.app`).
- RESUELTO 5. No existía eas.json**
Creado a mano en `APPS/wallet-control/eas.json` con perfiles `development` , `preview` (APK interno) y `production` (App Bundle), validado como JSON correcto. Falta solo el paso manual de `eas login / eas init` del usuario para obtener el `projectId` .
- RESUELTO 6. expo-notifications como plugin**
Declarado en `plugins` dentro de `app.json` , y ya en uso real (recordatorios de tarjetas, presupuesto y gastos recurrentes).

RESUELTO 7. Assets rotos

Los 6 archivos de ícono (icon, splash, adaptive-icon foreground/background/monochrome, favicon) eran la plantilla genérica de Expo — confirmado visualmente uno por uno. Se generó un ícono propio de billetera por código (paleta ya definida en `app.json` : `#0A0E1A` / `#E6F4FE`) y se reemplazaron los 6 archivos.

Plan a versión 1.0 — Fase 1 (checklist)

- ✓ Mover la llamada a Finando IA a un backend/proxy — *Supabase Edge Function*
- ✓ Eliminar el campo password
- ✓ Agregar bundleIdentifier / package y el plugin de expo-notifications
- ✓ Correr eas build:configure para generar eas.json — *creado a mano, sin requerir login*
- ✓ Re-exportar íconos y splash sin las guías de diseño — *ícono de billetera generado por código*
- ✓ Migrar a claude-sonnet-5

Pendiente manual del usuario (no bloquea el resto): correr eas login y eas init dentro de APPS/wallet-control para vincular el proyecto a una cuenta de Expo/EAS antes del primer eas build --profile preview.

Fase 2 — Pulido de producto (re-verificada y cerrada el 6 de julio)

#	Hallazgo	Archivo	Estado
8	Promesa de "Excel/CSV próximamente" que no existe	app/ayuda.tsx	RESUELTO
Reemplazada por un extracto bancario en PDF real (movimientos cronológicos con saldo corrido) — no solo se quitó el texto, se construyó la funcionalidad.			
9	Markdown del chat de Finando se muestra literal	components/ChatBubble.tsx	RESUELTO
10	QuincenaCard: stub vacío, código muerto	components/QuincenaCard.tsx	RESUELTO
Se construyó un sistema completo de periodicidad de cuenta (semanal/quincenal/mensual), no solo se llenó el componente — ver sección siguiente.			
11	BudgetProgressBar completo pero nunca montado	components/BudgetProgressBar.tsx	RESUELTO
12	Falta catch en el login	app/onboarding.tsx	RESUELTO
Re-verificado: handleLogin ya tenía try/catch/finally completo — no era un hallazgo vigente.			
13	No avisa si se deniega el permiso de notificaciones	components/CardFormModal.tsx	PENDIENTE
14	Versión desincronizada (v0.6.0 vs 1.0.0)	app/(tabs)/perfil.tsx	RESUELTO
Ahora usa Constants.expoConfig.version (expo-constants) en vez de texto hardcodeado.			
15	Sin tests automatizados	—	SIN CAMBIOS
16	Sin accessibilityLabel/Role en botones de solo-ícono	toda la app	PENDIENTE

Funcionalidades nuevas (no estaban en la auditoría original)

NUEVO Notificaciones de presupuesto

BudgetProgressBar + modal para fijar presupuesto mensual; alerta local al 80% y al 100% del gasto, sin repetirse en cada recarga.

NUEVO Gastos e ingresos recurrentes con recordatorio

Al registrar un gasto (o un ingreso fijo desde el onboarding) se puede marcar como recurrente (semanal/mensual); se agenda un recordatorio local a las 9AM y se cancela automáticamente si se borra el movimiento.

NUEVO Extracto bancario en PDF

Nueva exportación (junto al reporte financiero existente): movimientos del mes en orden cronológico con saldo acumulado, estilo estado de cuenta real.

NUEVO Periodicidad de cuenta

Nuevo paso en el onboarding: el usuario elige si maneja su dinero semanal, quincenal o mensualmente. Resumen muestra la tarjeta correspondiente (Semana / Quincena / Mes) con días restantes — sin alterar el sistema de quincenas ya usado para clasificar gastos.

NUEVO Ingreso y gasto fijo al crear cuenta (idea "MIAS")

Dos preguntas opcionales al final del onboarding (ingreso fijo mensual, gastos fijos mensuales); si se llenan, se crean como movimientos recurrentes reales desde el primer día, con recordatorio.

Fase 3 — Roadmap restante (NOTAS/ideas wallet-control.txt)

Alta prioridad (v1.0): Widget de saldo rápido en pantalla de inicio.

Media prioridad (v1.1): Búsqueda de gastos (por nombre/categoría/fecha), Comparativa mes a mes ("vs mes anterior").

Idea propia (MIAS): apartado de "inversionista" con consejos premium según perfil financiero.

Baja prioridad / v2.0: modo pareja/familia, categorización automática por IA, proyección de fin de mes, integración bancaria por lectura de extractos, versión web de solo lectura.

Cómo empezar a compartirla y conseguir audiencia

- 1 Build compatible sin fricción.**
Con la Fase 1 realmente cerrada, correr `eas login + eas init` (paso manual pendiente) y luego `eas build --platform android --profile preview` genera un APK instalable directo, compatible por link a un grupo cerrado de confianza (5-10 personas).
- 2 Beta cerrada (2-3 semanas).**
Recoger feedback específico (¿usan Finando IA?, ¿entienden el modo anónimo?, ¿qué confunde del onboarding con las nuevas preguntas de periodicidad e ingresos fijos?) antes de exponerse a audiencia más grande. En paralelo, subir a Google Play Internal Testing (gratis).
- 3 Fachada pública en paralelo.**
Landing de una sola página con la skill PAKI: qué hace la app, 2-3 screenshots reales, botón de descarga o lista de espera.
- 4 Lanzamiento público.**
Play Store (cuenta de desarrollador, \$25 único pago) + Product Hunt + comunidades de finanzas personales en español.
- 5 Ciclo de audiencia continuo.**
Usar el historial de ACTUALIZACIONES.txt como base para publicar novedades cada vez que se agregue algo del roadmap.

Verificación

- ✓ Fase 1 completa: código listo. *Falta solo eas login/init (manual, del usuario) antes del primer build real*
- ✓ Fase 2 completa: todos los hallazgos accionables resueltos o re-verificados
- Fase 2 (prueba de humo, cuando el usuario la corra en el servidor Expo): onboarding con periodicidad + ingreso/gasto fijo → chat con markdown → extracto bancario en PDF → tarjeta de período en Resumen
- Fase 4: confirmar que al menos 3-5 personas fuera del creador instalaron y usaron la app desde el APK compartido antes de subir a Play Store